

Lecture 13.1: Introduction to Context-free Grammar - 1

Presenter: Azwad Anjum Islam (AAI)

Scribe: Mujtahid Al-Islam Akon (AKO)

In this lecture, you will learn about the limitations of Regular Language and introduce you to a new class of language and its corresponding description method formally called the Context-free Grammar.

A limitation of Finite Automata

We know that every regular language has an equivalent finite automaton (e.g. DFA or NFA). They are called finite because they have a **very limited memory** in the form of states.

Though many interesting problems can be solved by Finite Automata, Finite Automata has a serious limitation –

A Finite Automaton does not have a large (infinite) memory. As a result, a finite automaton can only "count" (that is, maintain a counter, where different states correspond to different values of the counter) a finite number of input scenarios.

For example, there is no finite automaton that recognizes the following languages:

- $L = \{w \mid w = 0^n 1^n, n \geq 1\}$ = The set of all binary strings consisting of a number of a's followed by equal number of b's.
- $L = \{w \mid w = n_0(w) = n_1(w), n \geq 1\}$ = The set of all binary strings consisting of an equal number of 1's and 0's
- The set of strings over '(' and ')' that have "balanced" parentheses.

Context-free Grammar

A context-free Grammar (**CFG**) is an entirely different formalism for describing a class of languages called Context-free Language (**CFL**). It is more powerful than Regular expression and finite automaton in the sense that it can describe more languages than finite automaton.

- In a context free grammar, we express a language by a set of rules (called **production rules**). These rules describe how a string of the language can be generated.
- Consider the language, $L = \{w \mid w = 0^m 1^{2m}; m \geq 0\}$. This denotes the set of all binary strings consisting of a number of a's followed by twice the number of b's as a. This language can be expressed by the following CFG:

$$S \rightarrow 1S00$$

$$S \rightarrow \epsilon$$

Each expression of the above is called a **production rule**.

In the next lecture, we shall learn more about CFG.

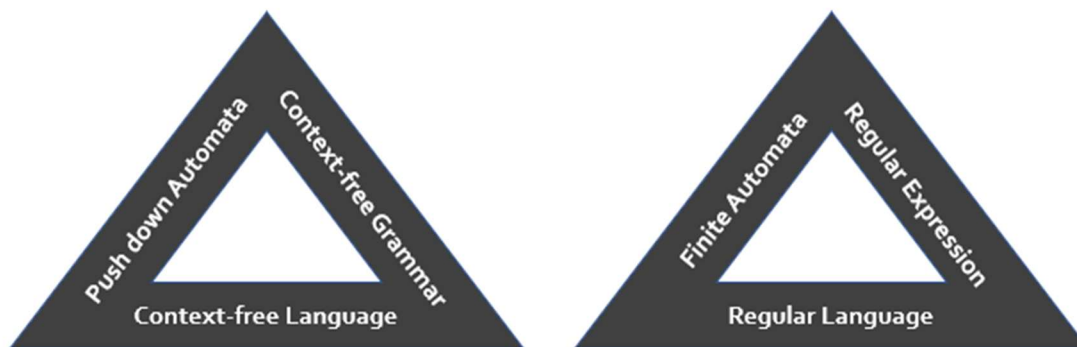
Lecture 13.2: Introduction to Context-free Grammar - 2

Presenter: Azwad Anjum Islam (AAI)

Scribe: Mujtahid Al-Islam Akon (AKO)

In this lecture, you will learn about more about the Context-free Grammar and how they work.

- We know that any regular language can be described by a method of description called Regular Expression. Similarly, any context-free language (CFL) can be described by a method of description called Context-free Grammar (CFG).
- We know that for any regular language, there exists an abstract machine that recognizes that particular language called Finite Automaton (FA). Similarly, for any CFL, there exists an abstract machine that recognizes that particular language called Pushdown Automaton (PDA).
- The following triangles portray the above relations-



Context-free Grammar

In a context free grammar, we express a language by a set of rules (also called **productions**). These rules describe how a string of the language can be generated. This can be best illustrated using the following example. Suppose, the following production rules are given for a language:

$$1. S \rightarrow 0S1$$

$$2. S \rightarrow \epsilon$$

Let us show how to generate a string using the rules given. Start with first symbol S , then replace any symbol with the right-hand-side of a rule. Continue this until no symbol appears.

S	(Start with S)
$\rightarrow 0S1$	(Replace with $0S1$ using rule-1)
$\rightarrow 01$	(Replace S with ϵ using rule-2)

So, we get 01 .

Let's try again.

S (Start with S)
 $\rightarrow 0S1$ (Rule-1)
 $\rightarrow 00S11$ (Rule-1 again)
 $\rightarrow 0011$ (Rule-2)

This time we get, 0011.

Is there any pattern arising? Let's try for the last time.

S (Start with S)
 $\rightarrow 0S1$ (Rule-1)
 $\rightarrow 00S11$ (Rule-1 again)
 $\rightarrow 000S111$ (Rule-1 once more)
 $\rightarrow 000111$ (Rule-2)

Here we get 000111.

Now, it is obvious that the above grammar (CFG to be specific) is generating only the set of strings consisting of some 0's followed by equal number of 1's. So, the above two production rules i.e. the above CFG, generating the language, $L = \{w | w = 0^n 1^n, n \geq 0\}$. In previous lectures, we learnt that this language is not regular. So, it can't be described by either a regular expression or a finite automaton.

Context free grammar notations

We have already seen that a context free grammar is expressed by a set of rules. Beside the rules, there are some implicit notations that you need to understand:

- The symbols that appear on the left side of a rule are called **non-terminals** or **Variables**. The non-terminal symbol needs to be replaced. There can't be any non-terminal symbol in the final string because these symbols are not a member of the alphabet Σ .
- The symbols that appear only on the right side and never appear on the left side of a rule are called **terminal** symbols. The terminal symbols are never replaced as they are not present on the left side of a rule. The terminal symbols are members of Σ . The final string generated consists of only terminal symbols.
- The non-terminal symbol that appears on the left side of the first rule is called the **start symbol** of the grammar. The production of a string will start with this symbol (see our examples above).